PROTOTSRL: INTERPRETABLE TIME SERIES REPRESENTATION LEARNING

Haiyan Wang
Duke University
haiyan.wang916@duke.edu

ABSTRACT

1 Introduction

Time series are prevalent across a wide range of domains and often drive decisionmaking in high-stakes environments including health monitoring, weather forecasting, and financial modeling. However, time series data pose significant challenges for human interpretability due to their high dimensionality with respect to both the length of the series and the number of features in each observation. Moreover, real-world time series data often present with irregularities such as temporal misalignment, uneven lengths, and variable sampling frequencies which can further obfuscate important structure. As a result, time series data are often sparsely labeled, and it is infeasible to annotate large datasets to the extent required for supervised modeling. The ability to learn fixed-length representations of time series without explicit supervision that differentiate relevant patterns from nuisance variation is important for both identifying latent structure within time series data as well as for downstream tasks such as classification and forecasting.

In this paper, we propose ProtoTSRL, a new approach to learning informative, general-purpose representations of time series that is both *interpretable* and *domain-agnostic*. Our primary contributions are:

- A contrastive learning framework that incorporates insights from dimension reduction algorithms by introducing a third class of contrasting samples separate from positive and negative pairings. Unlike previous approaches, contrastive samples are created with both sampling- and augmentation-based methods and do not rely on domain-specific assumptions such as local smoothness.
- A novel architecture that employs multiscale prototype-based learning to capture both local morphological traits and global temporal dynamics. The proposed architecture can be trained in stages, allowing us to shape semantic intermediate latent spaces which are then used to learn final representations.

2 Related Work

In the most general sense, representation learning refers to techniques that learn to extract and represent useful information from raw data. This is evidently a broad definition and many methods and models fall under this umbrella including clustering, dimension reduction, and neural networks. Here, we review methods for learning representations without explicit supervision, and in particular, on self-supervised methods. Prior work indicates that learning to solve appropriately designed auxiliary tasks based on data-derived pseudolabels induces models to represent important semantic information embedded in unlabeled data [1, 2, 3]. This is of particular importance for time series data which frequently lack human-recognizable attributes but yield well to pseudolabeling.

A common auxiliary task is learning to reverse augmentations. This is a particularly popular approach for visual data and computer vision (CV) – augmentations include decolorization [4], masking [5], and scrambling [6]. In this case, the original image or some subpart of it is used as the pseudolabel to guide learning, and such approaches assume that data may be characterized by semantic information that is invariant under augmentation. Other auxiliary tasks are contrastive (e.g. [7, 8]) – rather than learning to extract information that characterizes samples, these models learn to extract information that differentiates samples. When applied to downstream tasks such as classification, object detection, and segmentation, the learned representations match or outperform fully supervised training from scratch [2]. Representation learning has also been successful in settings that involve sequential data. In natural language processing (NLP), algorithms like Word2Vec [9] learn static embeddings for words based on the contexts in which each word was found throughout a large training corpus. Subsequent developments such as Doc2Vec [10] learned fixed-length representations of entire variable-length passages. Such representations are applied in downstream tasks like topic modeling [11] and machine translation [12]. The invention of the transformer architecture [13] further advanced the ability to generate contextual representations via models like BERT [14]. The auxiliary tasks learned by these models are simple predictive problems such as masked- or next- token prediction. Similar techniques (e.g. [15, 16, 17]) are applied to tasks involving audio data despite its higher complexity relative to language: it is significantly higher dimensional, continuous, and contains many entangled components that don't yield well to segmentation [3].

The success of representation learning across such a wide range of data modalities suggests it may be well-suited for application to time series data. Broadly speaking, there are two types of representations that can be learned for a multivariate time series: instance-level representations capture entire series, encompassing multivariate observations across multiple timestamps, while semantic-level representations capture each timestamp individually. The primary focus of existing literature is designing appropriate auxiliary tasks. Some of these tasks are borrowed directly from other fields: for example, drawing inspiration from BERT, encoder-only transformers have learned effective representations by reconstructing masked observations or subsegments within a longer time series [18]. SOM-VAE [19] learns representations at a semantic level and opts for a reconstructive auxiliary task: a variational autoencoder (VAE) learns a semantic latent space that is shaped by a discrete self-organizing map (SOM). Given a multivariate time series, the observation at each timestamp is mapped to a continuous latent vector which is then assigned to its nearest SOM embedding. The decoder reconstructs the observation from both the continuous embedding and its SOM label. To represent temporal dynamics, a Markov transition model is learned over the discrete SOM states to predict the state of the next observation in the series.

Other prior methods implement contrastive auxiliary objectives, raising the question of how best to define contrastive examples. Two major approaches are reflected in the literature: sampling and augmentation. Sampling is usually time-based (e.g. [20]) and draws inspiration from previous successes in NLP. Time-based sampling assumes that, given an arbitrary subseries sampled from a larger time series, temporally close subseries should have similar representations while distant subseries (temporally distant in the same series or from another series) should have different representations. An obvious weakness of time-based sampling is the strong assumption of local smoothness which may not hold for series with sudden distribution changes. TNC [21] addresses this with a weaker assumption of local smoothness by drawing positive examples for a given subseries from a normal distribution centered at that subseries, the variance of which is a tunable hyperparameter. Additionally, TNC employs positive-unlabeled learning, a variant of contrastive learning which treats nonpositive samples as unknown and reweights their contribution to the overall contrastive loss. The other approach to defining contrastive examples is via augmentation. TS-TCC [22] creates different views of the same series with weak (scaling and jittering) and strong (permutation and jittering) augmentations. These views are encoded to an intermediate latent space shaped by a temporal contrasting objective based on cross-view prediction. Intermediate embeddings are then mapped by a nonlinear function to the final representation space which is shaped by a standard contrasting objective. This framework encourages the model to learn both temporal and discriminative patterns that are preserved under augmentation. Lastly, TS2Vec [23] also learns semantic-level representations, but these representations can be aggregated into instance-level representations. TS2Vec defines positive pairs by sampling overlapping subseries which are randomly masked and cropped to form distinct contexts; overlapping timestamps are taken as positive pairs. The model learns via a loss function that incorporates temporal and instance-wise contrast – the former contrasts representations of the same timestamp while the latter contrasts representations of the same series. To move from semantic- to instance-level representations, TS2Vec repeatedly aggregates its representations and computes this loss over the aggregations. This procedure, termed hierarchical contrasting, encourages the model to learn representations that are durable under aggregation to multiple temporal scales.

Interpretable Representation Learning The primary disadvantage of these methods is the lack of interpretability: representations are generated by complex nonlinear mappings from raw data to a high-dimensional space. In some cases, the structure of the representation space is simple, but we have no idea why the model learns this structure (e.g. SOM-VAE). In other cases, we get lucky and the structure of the representation space reflects a natural or human-interpretable semantic structure (e.g. word2vec), but we often discover this fact through *post-hoc* analysis: explainability is rarely a consequence of intentional design choices. In general, the representations learned by these methods are sufficiently *rich* to drive downstream tasks, but they are hardly *informative* to humans. This is undesirable because representations generated by a black box preclude any possibility for interpretability downstream, and in many cases, we may wish to use the learned representations to identify meaningful patterns or structures in our original data.

Interpretability is especially challenging to achieve with multivariate time series because interactions between features over time are rarely discernible to begin with, and this additional complexity also requires more complex models to capture. A useful starting point for designing interpretable models is to consider how humans would reason about the problem. Take for example heart rate data measured by photoplethysmogram (PPG) and classification of atrial fibrillation: physicians often search for patterns such as fluctuating pulse frequency and inconsistent pulse shapes [24]. The human intuition of searching for recognizable morphological patterns embedded within a larger time series suggests an approach involving prototype learning where a model learns representative examples of concepts (prototypes) and reasons by comparing new inputs to these examples. In practice, this reasoning process is implemented as a clustering task within a learned latent space [25]. Prototype learning has been successfully applied to CV for classification and segmentation [26, 27, 28], but it is not limited to visual data. In particular, the success of ProtoViT [29] suggests that meaningful prototypes can be learned for sequential data as long as the latent space is defined by an appropriate encoder backbone that accounts for sequentiality (in this case a vision transformer). Conceptually, a prototype-based approach to learning time series representations is most similar to SOM-VAE in the way the latent space is shaped by a discrete set of points, but the learned SOM vertices are not interpretable.

Dimension Reduction A related but distinct field to representation learning is dimension reduction (DR). While representation learning aims to produce informative (and often high-dimensional) embeddings of raw data that are useful for a variety of downstream tasks, DR is primarily used to generate low-dimensional embeddings for visualization that preserve the geometric and topological structure of data. As a result, DR is not well-suited for the task of representation learning and vice versa. This is especially true with respect to time series where factors such as uneven length, irregular sampling, and temporal dependence complicate the definition of "distance" between two series. However, recent advances in DR offer valuable insights into how loss function design affects the geometry of learned embedding spaces. In particular, embedding quality depends on both the notion of similarity that is encoded as well as how the loss allocates attractive and repulsive forces across samples. Well-designed objectives preserve local neighborhoods without sacrificing global organization by balancing attraction to close points with carefully controlled repulsion from farther ones. PaCMAP [30] shows that applying forces to non-neighbors is key to maintaining this balance and prevents embeddings from collapsing into locally coherent but globally misleading clusters. These insights may be beneficial for broader representation learning objectives: rather than treating all nonpositive pairs as uniformly negative, a contrastive model may benefit from distinguishing truly dissimilar series from samples that are not nonpositive but share some temporal or morphological structure because those intermediate relations help organize the embedding space at a global level.

3 Methods

Our overarching goal is to learn fixed-length representations of variable-length, potentially multivariate time series without explicit supervision. These representations need to be sufficiently rich to drive downstream tasks while remaining interpretable – in particular, we want to be able to understand which morphological and temporal characteristics of the original series are used by the model to shape the representation space. Section 3.1 describes our model architecture which implements a multiscale convolutional encoder with sequential prototype layers of different receptive fields to capture local morphological traits as well as global temporal dynamics. Section 3.2 describes our contrastive learning framework which incorporates insights from DR by introducing a third class of contrasting samples separate from positive and negative pairings to organize the representation space.

3.1 Model Architecture

Figure 1 summarizes our model architecture. We use a multiscale convolutional encoder f composed of sequential segments $\mathcal{S}$ where each segment consists of one or more 1D convolutional layers. Early segments use dense or weakly dilated convolutions to capture local morphological details while deeper segments dilate exponentially to expand the receptive field and encode long-range temporal dynamics. Let $f_{\theta_i}^{(i)}$ denote the i^{th} segment of the convolutional encoder parametrized by θ_i, and let $f_{\theta_{\text{conv}}}$ denote the entire convolutional encoder parametrized by θ_{conv} with L segments:

$$f_{\theta_{\text{conv}}} = f_{\theta_L}^{(L)} \circ \cdots \circ f_{\theta_2}^{(2)} \circ f_{\theta_1}^{(1)}$$

Unlike a standard prototype network with a single prototype layer at the end of the encoder, we insert prototype comparisons between encoder segments. Let $x \in \mathbb{R}^{T \times F}$ be a potentially multivariate time series. Denote the latent embedding after segment $\ell \in \mathcal{S}$ by $z_\ell(x) \in \mathbb{R}^{T_\ell \times F_\ell}$ where

$$z_\ell(x) = f_{\theta_\ell}^{(\ell)} \circ \cdots \circ f_{\theta_1}^{(1)}(x)$$

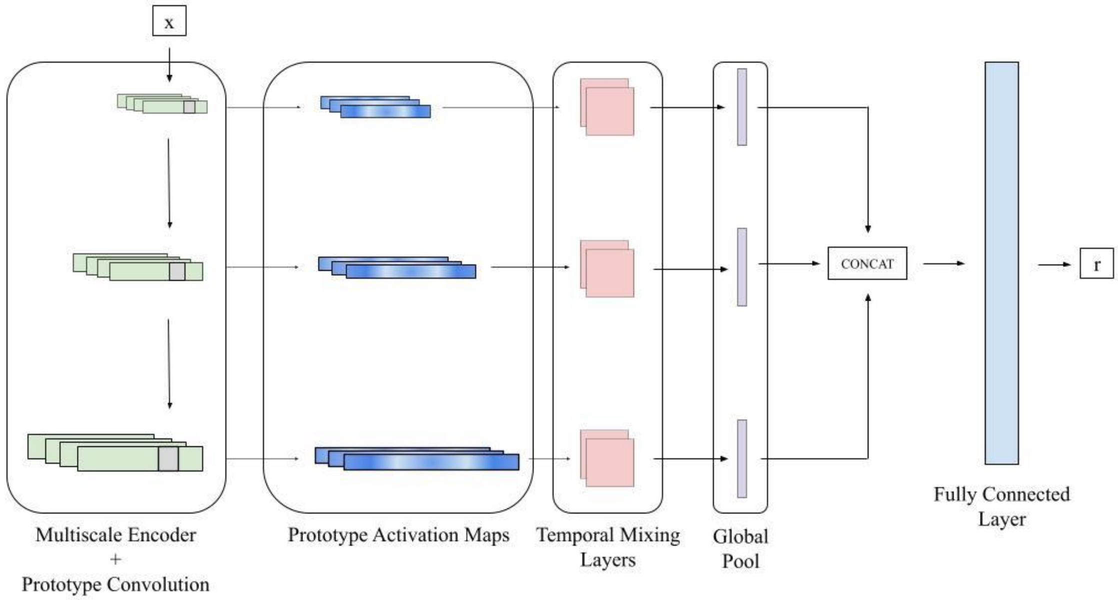

Figure 1: Generic ProtoTSRL architecture.

We attach a prototype bank $P_\ell = \{p_1^{(\ell)}, \dots, p_{M_\ell}^{(\ell)}\}$ after the ℓ^{th} segment where each prototype $p_m^{(\ell)} \in \mathbb{R}^{W_\ell \times F_\ell}$ is a learnable patch in the space of z_ℓ with $W_\ell \leq T_\ell$. The shallow prototype banks are intended to capture local morphological patterns, while deeper prototype banks capture longer-range temporal motifs in more contextual latent spaces. This yields a sequential prototype model in which different encoder depths correspond to different temporal scales of explanation.

For each prototype $p_m^{(\ell)}$, we compute its inverted cosine similarity to each patch in $z_\ell(x)$, forming an activation map $a_m^{(\ell)}(x) \in \mathbb{R}^{T_\ell}$. In another deviation from a standard prototype network, we do not perform global max pooling over $a_m^{(\ell)}(x)$. Instead, the model preserves the full activation map of every prototype and learns the final representation from the temporal evolution of these prototype responses. This lets the representation retain information not only about whether a pattern occurs, but also about how prototype evidence is organized across time and across scales. Collecting the activation maps of all prototypes at that depth gives the prototype activation tensor:

$$A^{(\ell)}(x) = \begin{bmatrix} a_1^{(\ell)}(x) \\ \vdots \\ a_{M_\ell}^{(\ell)}(x) \end{bmatrix} \in \mathbb{R}^{M_\ell \times T_\ell}$$

We reiterate that because deeper prototype layers operate on features with larger receptive fields, activations in later layers already reflect similarity to longer subseries in the original input. Thus, temporal dynamics are represented jointly by the encoder's multiscale receptive fields and by the time-varying prototype activation maps.

For each prototype layer ℓ, the prototype activation tensor $A^{(\ell)}(x)$ is treated as a multichannel temporal signal whose channels correspond to learned prototypes. We then apply a temporal convolution module $g_\psi^{(\ell)} : \mathbb{R}^{M_\ell \times T_\ell} \to \mathbb{R}^{C_\ell \times T'_\ell}$ to the activation tensor where C_ℓ is the number of output channels for that layer and T'_ℓ is the resulting length. A separate temporal convolution module is used for each prototype layer to accommodate differences in prototype count, temporal resolution, and receptive-field scale across depths. The purpose of this module is to learn temporal patterns over prototype responses. Since the input channels are prototype activation maps, the temporal convolution mixes across both the channel dimension and the temporal dimension. Therefore, each output channel can capture patterns such as co-activation of multiple prototypes, short-range ordering of prototype responses, repeated prototype activity, or scale-specific temporal motifs expressed in the prototype space. Because the convolution operates on prototype maps rather than on uninterpretable hidden features, the resulting temporal features remain grounded in prototype-based evidence. Lastly, to obtain a single-channel output from layer ℓ, we apply an adaptive pooling operation over the temporal dimension which we denote as $\text{Pool}_\ell : \mathbb{R}^{C_\ell \times T'_\ell} \to \mathbb{R}^{C_\ell}$, yielding

$$b^{(\ell)}(x) = \left(\text{Pool}_\ell \circ g_\psi^{(\ell)} \circ A^{(\ell)}\right)(x)$$

We concatenate the outputs of all prototype layers:

$$u(x) = \left[b^{(1)}(x); \ldots; b^{(L)}(x)\right]$$

A linear projection head maps $u(x)$ to the final representation:

$$h(x) = Wu(x) + b$$

3.2 Contrastive Learning Framework

Sampling Let $x^{(i)} \in \mathbb{R}^{T_i \times F}$ denote a potentially multivariate time series and let $x_{\text{anc}}^{(i)}, x_{\text{pos}}^{(i)}, x_{\text{mid}}^{(i)} \subset x^{(i)}$ be temporally contiguous subseries of $x^{(i)}$ where $x_{\text{anc}}^{(i)}$ and $x_{\text{pos}}^{(i)}$ are overlapping and $x_{\text{mid}}^{(i)}$ is distinct from $x_{\text{anc}}^{(i)}$ and $x_{\text{pos}}^{(i)}$. We independently perturb each of these subseries via jittering (prior to encoding) and masking (after the first encoder segment) to produce related contexts $\tilde{x}_{\text{anc}}^{(i)}, \tilde{x}_{\text{pos}}^{(i)}, \tilde{x}_{\text{mid}}^{(i)}$. We consider $\tilde{x}_{\text{anc}}^{(i)}$ to be the anchor, $\tilde{x}_{\text{pos}}^{(i)}$ the positive example, and $\tilde{x}_{\text{mid}}^{(i)}$ the mid-near example (i.e. it is less similar to the anchor than an overlapping view, but it is still more related than a sample drawn from a different series). This intermediate relation is inspired by PaCMAP's distinction between neighbors, mid-near points, and further points, which is used there to balance local and global geometric structure. Here, the same idea is adapted to contrastive learning where intermediate pairs are used to improve the model's learned notion of similarity. Negative examples are sampled from other time series in the minibatch during training and denoted $\tilde{x}_{\text{neg}}^{(i,j)}$ where $j \neq i$ denotes the series from which it was sampled. Note that while examples do not have to be the same length, care must be taken to ensure that the dimensionality of intermediate embeddings is compatible with prototype layers. In this work, all examples within a batch are the same size for code simplicity, but batches themselves are constructed with samples of different lengths to ensure that the model learns robust representations across different series lengths.

Prototype Learning The first role of training is to shape the prototype spaces so that related subseries induce similar prototype responses. Because the model preserves the full activation map of each prototype, prototype similarity is defined at the level of these activation tensors rather than only pooled scalar scores. Let $s(x, y)$ be the cosine similarity between vectors x and y:

$$s(x, y) = \frac{\langle x, y \rangle}{\|x\|_2 \|y\|_2}$$

As before, let $A^{(\ell)}(x)$ be the prototype activation tensor of input x at layer ℓ. We define the ℓ-prototype-space similarity between two series x and y to be the cosine similarity between their flattened prototype activation tensors $\overline{A}^{(\ell)}(x)$ and $\overline{A}^{(\ell)}(y)$:

$$s_{\text{proto}}^{(\ell)}(x, y) = s\left(\overline{A}^{(\ell)}(x), \overline{A}^{(\ell)}(y)\right)$$

Two subseries are similar in a prototype space when they induce similar time-varying prototype activation patterns. Using this similarity, we define pair-specific losses for positive, mid-near, and negative relations:

$$\mathcal{L}_{\text{pos}}(s) = 1 - s, \qquad \mathcal{L}_{\text{mid}}(s) = 0.5(1 - s), \qquad \mathcal{L}_{\text{neg}}(s) = \max(0, s - 0.1)$$

Overall, for a training example $x^{(i)}$ from which we sample the anchor $\tilde{x}_{\text{anc}}^{(i)}$, a positive example $\tilde{x}_{\text{pos}}^{(i)}$, a mid-near example $\tilde{x}_{\text{mid}}^{(i)}$, and negative examples $\tilde{x}_{\text{neg}}^{(i,j)}$ where$j \neq i$ and N_i indexes the minibatch, the loss for layer ℓ is

$$\mathcal{L}_{\text{proto}}^{(\ell)}(x^{(i)}) = \mathcal{L}_{\text{pos}}\left(s_{\text{proto}}^{(\ell)}\left(\tilde{x}_{\text{anc}}^{(i)}, \tilde{x}_{\text{pos}}^{(i)}\right)\right) + \mathcal{L}_{\text{mid}}\left(s_{\text{proto}}^{(\ell)}\left(\tilde{x}_{\text{anc}}^{(i)}, \tilde{x}_{\text{mid}}^{(i)}\right)\right) + \frac{1}{|N_i|}\sum_{j \in N_i} \mathcal{L}_{\text{neg}}\left(s_{\text{proto}}^{(\ell)}\left(\tilde{x}_{\text{anc}}^{(i)}, \tilde{x}_{\text{neg}}^{(i,j)}\right)\right)$$

Because prototypes within the same layer should represent distinct latent components, we additionally include a diversity regularizer that penalizes prototypes that are too similar to one another. This penalty discourages prototype collapse and encourages each prototype bank to span a diverse set of prototypical temporal components. Let $p_a^{(\ell)}$ and

$p_b^{(\ell)}$ denote two distinct prototypes in layer ℓ (flattened if needed), and define their similarity as the cosine similarity between them:

$$r_{\text{proto}}^{(\ell)}(a, b) = s\left(p_a^{(\ell)}, p_b^{(\ell)}\right)$$

The overall prototype diversity penalty for layer ℓ is defined as the average pairwise similarity across all prototypes:

$$\mathcal{R}_{\text{div}}^{(\ell)} = \frac{1}{M_\ell(M_\ell - 1)} \sum_{a \neq b} \max\left(0, r_{\text{proto}}^{(\ell)}(a, b) - 0.2\right)$$

Over a batch B, the objective across all prototype spaces is

$$\mathcal{L}_{\text{proto}} = \frac{1}{|B|} \sum_{i \in B} \sum_{\ell \in \mathcal{S}} \mathcal{L}_{\text{proto}}^{(\ell)}(x^{(i)}) + \lambda_{\text{proto}} \sum_{\ell \in \mathcal{S}} \mathcal{R}_{\text{div}}^{(\ell)} \tag{1}$$

Representation Learning After the temporal convolution modules transform the prototype activation maps into fixed-length descriptors, the second role of training is to learn a final representation that preserves the same positive/mid-near/negative ordering. Let $h(x) \in \mathbb{R}^d$ denote the final representation produced by the linear head. We again use cosine similarity:

$$s_{\text{repr}}(x, y) = s\left(h(x), h(y)\right)$$

In contrast to the prototype-space objective, we use a modified exponential loss to shape the representation space:

$$\mathcal{L}_{\text{repr}}(x^{(i)}) = -\log\left[\frac{\exp\left(s_{\text{repr}}\left(\tilde{x}_{\text{anc}}^{(i)}, \tilde{x}_{\text{pos}}^{(i)}\right)\right)}{\exp\left(s_{\text{repr}}\left(\tilde{x}_{\text{anc}}^{(i)}, \tilde{x}_{\text{pos}}^{(i)}\right)\right) + \sum_{j \in N_i} \exp\left(s_{\text{repr}}\left(\tilde{x}_{\text{anc}}^{(i)}, \tilde{x}_{\text{neg}}^{(i,j)}\right)\right)}\right] + 0.5\left[1 - s_{\text{repr}}\left(\tilde{x}_{\text{anc}}^{(i)}, \tilde{x}_{\text{mid}}^{(i)}\right)\right]$$

The exponential loss in the first term is a common choice for contrastive learning (e.g. [22, 23]), and in the vanilla contrastive approach where we only consider positive and negative examples, optimizing this loss maximizes a lower bound on mutual information between paired views [7]. The second term adds a weaker attractive constraint for the mid-near sample rather than treating it as a negative competitor. This modification does not maintain the mutual-information bound for the entire objective, but it preserves the intended geometry of the representation space: positives are pulled together strongly, negatives are contrasted repulsively through the softmax denominator, and mid-near samples are encouraged to remain similar but under a looser constraint.

Since the final representation is generated by a linear layer, we additionally impose L_1 regularization on the linear weights to control complexity and discourage over-reliance on a small subset of prototype-derived features. The full representation-space objective over a batch B is therefore

$$\mathcal{L}_{\text{repr}} = \frac{1}{|B|} \sum_{i \in B} \mathcal{L}_{\text{repr}}(x^{(i)}) + \lambda_{\text{repr}} \|W\|_1 \tag{2}$$

We shape the final representation space and the intermediate prototype spaces with different objectives because these stages serve different roles. The intermediate prototype losses are intended to organize the latent spaces around interpretable and diverse prototype activation patterns. This is a more complex structural goal than standard batch discrimination, particularly when a third class of contrasting examples (mid-near samples) are included, and exponential losses are not well-suited for this task due to their emphasis on hard negatives. By contrast, the final representation head is responsible for producing a compact embedding that is globally discriminative within the minibatch. At that stage, an exponential loss is more appropriate.

3.3 Training

We optimize the model in three stages, following the staged training philosophy of ProtoPNet while adapting it to our contrastive framework. ProtoPNet separates learning a meaningful prototype-structured latent space from optimization of the final layer; we adopt the same idea here, but with unsupervised positive, mid-near, and negative relations rather than labeled classes.

Stage 1 In the first stage, we train the encoder and all prototype banks. This can be understood as a pretraining stage guided by the optimization

$$\min_{\theta_{\text{conv}},\Phi} \mathcal{L}_{\text{proto}}$$

where $\Phi = \{P_\ell\}_{\ell\in\mathcal{S}}$ denotes all prototype parameters. This stage shapes the multiscale latent spaces so that related subseries produce similar prototype activation maps at each prototype depth, while the diversity regularizer encourages each prototype bank to learn a non-redundant set of prototypical temporal components.

Stage 2 In the second stage, we freeze the encoder and prototype layers and train the temporal convolution modules together with the final linear head, guided by the optimization

$$\min_{\Psi,W,b} \mathcal{L}_{\text{repr}}$$

where $\Psi = \{\psi^{(\ell)}\}_{\ell\in\mathcal{S}}$ denotes the parameters of the temporal convolution modules $\{g_\psi^{(\ell)}\}_{\ell\in\mathcal{S}}$. Because prototypes are fixed, this stage learns how to transform and aggregate multiscale prototype activation trajectories into a useful fixed-length representation without altering the prototype structure learned in Stage 1.

Stage 3 In the final stage, we unfreeze the entire network and jointly optimize all parameters:

$$\min_{\theta_{\text{conv}},\{P_\ell\}_{\ell\in\mathcal{S}},\Psi,W,b} \mathcal{L}_{\text{repr}}$$

This final fine-tuning stage allows the encoder, prototypes, temporal readout modules, and linear head to co-adapt while starting from a prototype-structured initialization.

4 Experiments

We train ProtoTSRL on several sets of univariate time series and evaluate the model's training dynamics and the quality of the learned representations. The geometry of the representation space can be visualized with DR (we use PaCMAP), and representation quality is evaluated with classification-based downstream tasks.

4.1 Synthetic Time Series

In this section, we create several synthetic datasets to test the model's ability to represent and distinguish broadly important characteristics of time series such as periodicity, stationarity, and temporal dependence. Each dataset consists of variable-length time series simulated by one of several user-defined generative processes. To ensure sufficient diversity within the generated data, we add small perturbations to the parameters of each generative process so that each sample is parametrized slightly differently but still clearly belongs to a specific class. The same model architecture is trained for each dataset. Importantly, this architecture outputs representations with 300 features and implements four encoder segments: the first two are dense, so their respective prototype layers learn localized features, while the last two dilate exponentially, so their prototype layers learn longer-term features. To evaluate the learned representations, we fit several models of varying complexity to recover the label of the original generative process.

Periodic Series This dataset consists of periodic (sinusoidal) series. The data generative process is determined by two components: wave parameters (frequency, amplitude, vertical shifts, phase shifts, drift) and noise parameters (amplitude jittering, frequency jittering, additive Gaussan noise, baseline wander). We generate five classes: (1) base sine wave, (2) base sine wave with different frequency and amplitude, (3) sine wave with positive linear drift, (4) sine wave with negative linear drift, (5) sine wave with positive vertical shift. The same noise parameters are applied to all 5 classes. Examples of a generated series from each class are shown in Figure 2.

We first analyze the evolution of the learned prototype spaces throughout training via the average pairwise similarity between prototypes in each layer. Lower average similarity implies the prototypes are more diverse and further apart within their respective latent space, thereby learning to capture a broader range of morphological structures. We expect prototype similarity to decrease in Stage 1 training due to the prototype regularization component and to remain constant in Stage 2 as prototype layers are frozen. However, in Stage 3, the objective does not regularize prototypes at all, so the behavior of prototype similarity is unknown. Prototype similarities at each layer are shown in Figure 3. Perhaps predictably, these trajectories indicate that the model quickly prefers to diversify later prototype spaces that represent

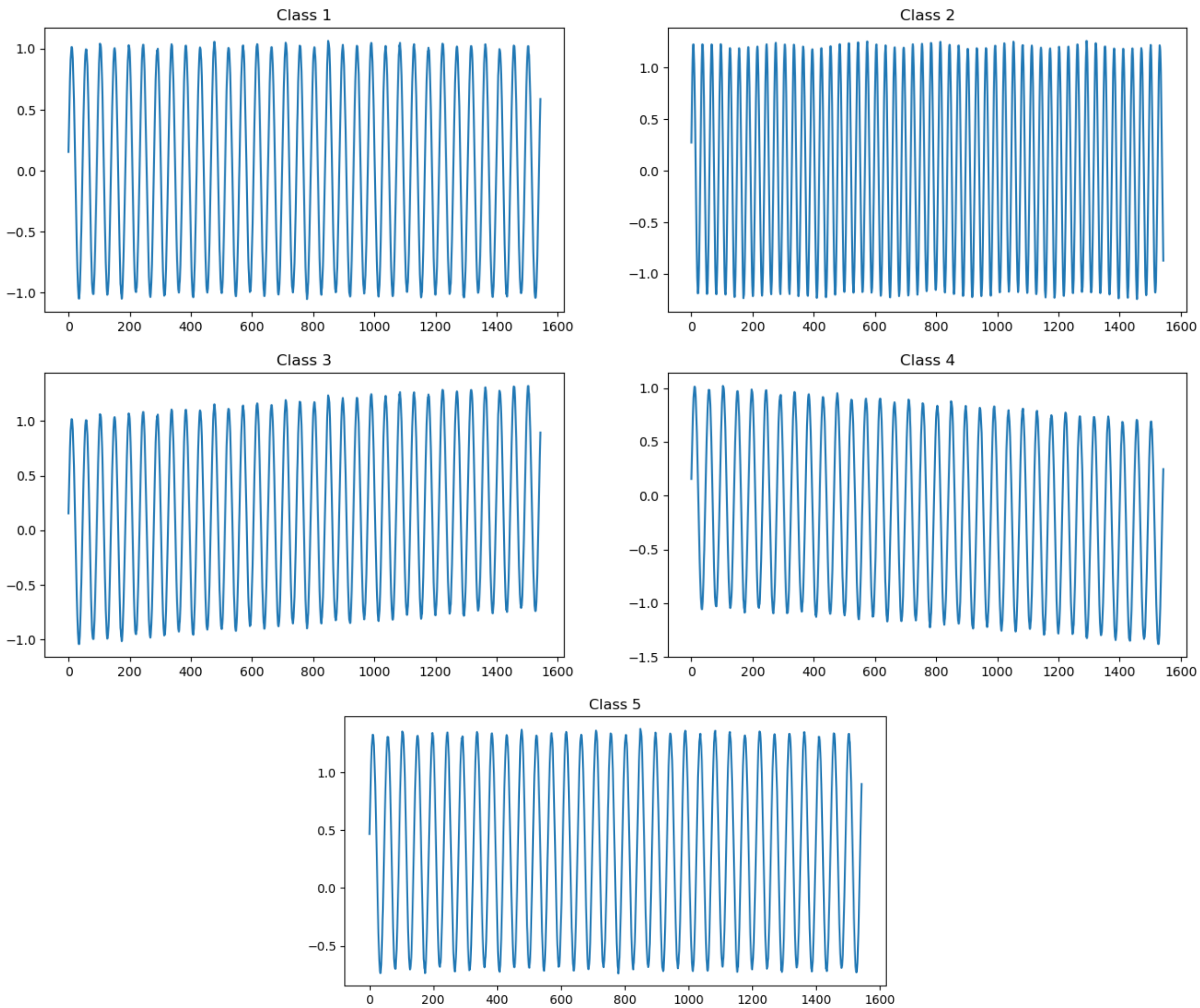

Figure 2: Example series from each class of periodic series.

long-term morphologies. Indeed, for highly periodic series such as these, local morphologies are unlikely to hold much unique information and are more prone to corruption due to added noise – all important characteristics of this dataset (i.e. wave parameters) are present and detectable over long time horizons. However, it appears that global training disrupts this paradigm: in final epochs, early prototype layers fluctuate significantly while later ones are largely fixed.

Because ProtoTSRL learns fixed-length representations, we are additionally able to visualize the representation space directly via DR and evaluate representation quality through the geometry of the learned space. PaCMAP embeddings of periodic series representations in two and three dimensions are shown in Figure 4. We include both two and three dimensional embeddings because the geometry of the embedding is not always obvious in two dimensions. These embeddings are also somewhat predictable. ProtoTSRL clearly distinguishes series by frequency and amplitude (classes 1 and 2). On the other hand, positive vertical shift and positive linear drift (classes 3 and 5) result in series that overlap extensively, and the model struggles to separate them. Also of note is the separation between the base series and the negative linear drift (classes 1 and 4). The two are separable, but their embeddings appear intertwined; this is particularly evident in three dimensions.

Lastly, we evaluate representation quality with a classification-based downstream task. We train models of varying complexities (logistic regression, decision tree, random forest, and multilayer perceptron) to label each representation with the generative process from which it was drawn. Model performance metrics are listed in Table 1.

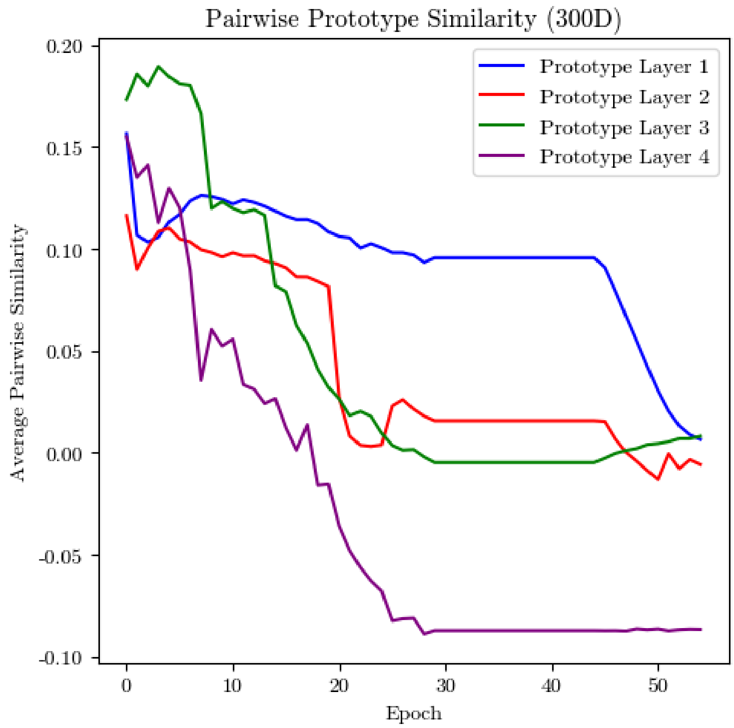

Figure 3: Average pairwise prototype similarity by layer.

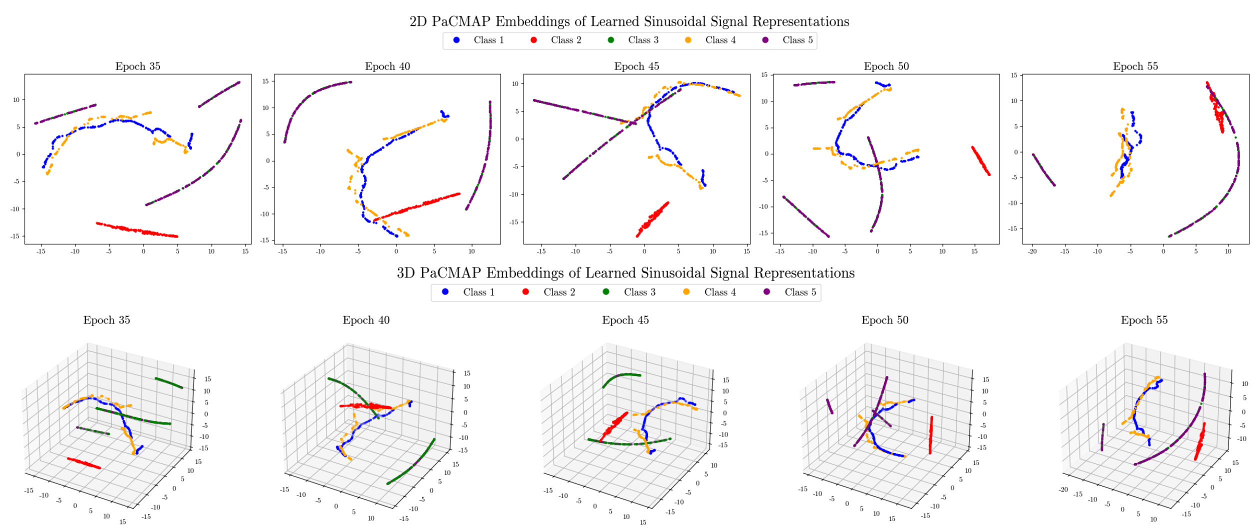

Figure 4: PaCMAP embeddings of periodic series representations in two and three dimensions.

	Logistic Regression	Decision Tree	Random Forest	Multilayer Perceptron
Accuracy	0.952	0.964	0.982	0.964

Table 1: Model accuracies: downstream classification.

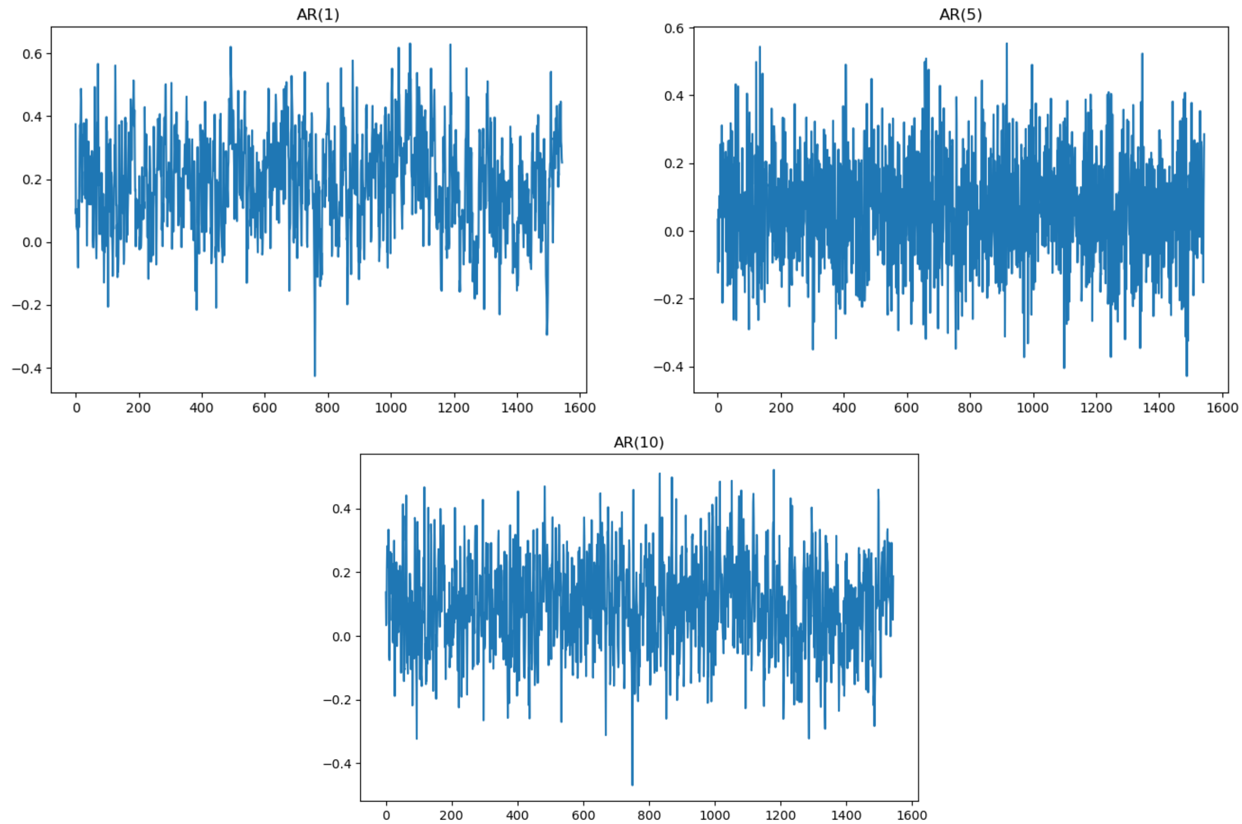

Figure 5: Example series from each class of autoregressive series.

Autoregressive Series This dataset consists of autogressive series of order p (AR(p)):

$$x_t = \beta_0 + \sum_{k=1}^{p} \beta_k x_{t-k} + \epsilon$$

where $\epsilon \sim N(0, \sigma^2)$. Each generative process is determined by the parameters p and σ^2. We enforce all series to be stationary, and we generate five classes with σ^2 constant: AR(1), AR(5), and AR(10). Examples of a generated series from each class are shown in Figure 5.

Prototype similarities at each layer are shown in Figure 6. Again, the model prefers to diversify later prototype spaces. Moreover, global training significantly disrupts learned prototypes in earlier layers.

PaCMAP embeddings of autoregressive series representations in two and three dimensions are shown in Figure 7. From this result, it seems ProtoTSRL is exceptionally good at learning to recognize temporal dependence structure as the representation space achieves perfect separation. This is reflected in downstream tasks as well: every model achieved perfect accuracy when predicting which autoregression generated each representation.

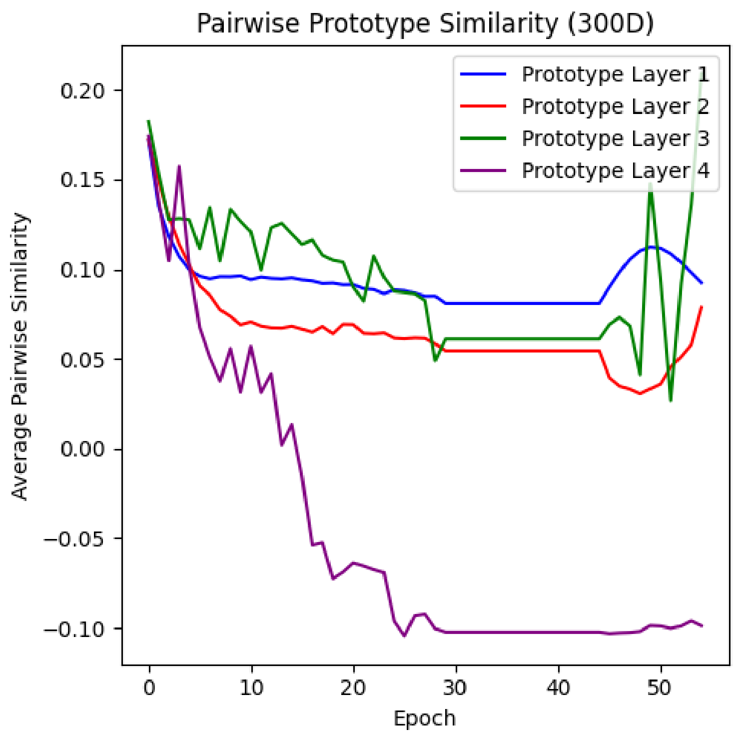

Figure 6: Average pairwise prototype similarity by layer.

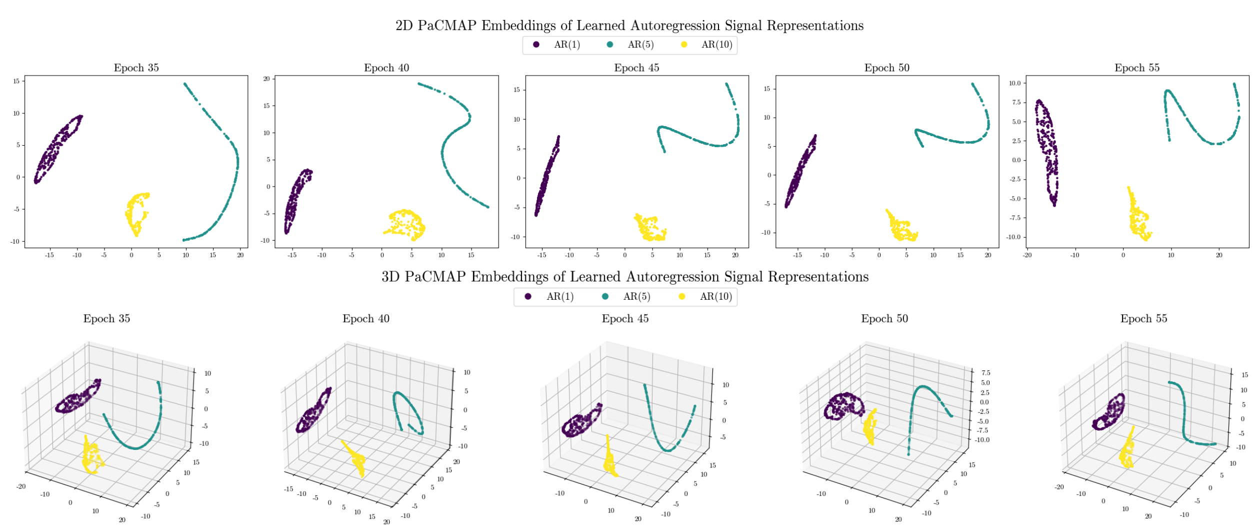

Figure 7: PaCMAP embeddings of autoregressive series representations in two and three dimensions.

4.2 PPG Signals

5 Discussion

References

[1] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation Learning: A Review and New Perspectives. *IEEE Trans. Pattern Anal. Mach. Intell.*, 35(8):1798–1828, August 2013.

[2] Longlong Jing and Yingli Tian. Self-Supervised Visual Feature Learning With Deep Neural Networks: A Survey. *IEEE Transactions on Pattern Analysis & Machine Intelligence*, 43(11):4037–4058, November 2021.

[3] Abdelrahman Mohamed, Hung-yi Lee, Lasse Borgholt, Jakob D. Havtorn, Joakim Edin, Christian Igel, Katrin Kirchhoff, Shang-Wen Li, Karen Livescu, Lars Maaløe, Tara N. Sainath, and Shinji Watanabe. Self-Supervised Speech Representation Learning: A Review. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1179–1210, 2022.

[4] Richard Zhang, Phillip Isola, and Alexei A. Efros. Colorful Image Colorization, 2016.

[5] Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell, and Alexei A Efros. Context encoders: Feature learning by inpainting. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2536–2544, 2016.

[6] Mehdi Noroozi and Paolo Favaro. Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles, 2017.

[7] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation Learning with Contrastive Predictive Coding, 2019.

[8] Yonglong Tian, Dilip Krishnan, and Phillip Isola. Contrastive Multiview Coding. In *Computer Vision - ECCV 2020: 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part XI*, pages 776–794, Berlin, Heidelberg, 2020. Springer-Verlag.

[9] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient Estimation of Word Representations in Vector Space, 2013.

[10] Quoc Le and Tomas Mikolov. Distributed Representations of Sentences and Documents. In Eric P. Xing and Tony Jebara, editors, *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*, pages 1188–1196, Bejing, China, 22–24 Jun 2014. PMLR.

[11] Dimo Angelov. Top2Vec: Distributed Representations of Topics, 2020.

[12] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2*, NIPS'14, pages 3104–3112, Cambridge, MA, USA, 2014. MIT Press.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *North American Chapter of the Association for Computational Linguistics*, 2019.

[15] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, pages 6309–6318, Red Hook, NY, USA, 2017. Curran Associates Inc.

[16] Yu-An Chung and James R. Glass. Generative Pre-Training for Speech with Autoregressive Predictive Coding. In *2020 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2020, Barcelona, Spain, May 4-8, 2020*, pages 3497–3501. IEEE, 2020.

[17] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: a framework for self-supervised learning of speech representations. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.

[18] George Zerveas, Srideepika Jayaraman, Dhaval Patel, Anuradha Bhamidipaty, and Carsten Eickhoff. A Transformer-based Framework for Multivariate Time Series Representation Learning. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, KDD '21, pages 2114–2124, New York, NY, USA, 2021. Association for Computing Machinery.

[19] Vincent Fortuin, Matthias Hüser, Francesco Locatello, Heiko Strathmann, and Gunnar Rätsch. SOM-VAE: Interpretable Discrete Representation Learning on Time Series. In *International Conference on Learning Representations*, 2019.

[20] Jean-Yves Franceschi, Aymeric Dieuleveut, and Martin Jaggi. *Unsupervised scalable representation learning for multivariate time series*. Curran Associates Inc., Red Hook, NY, USA, 2019.

[21] Sana Tonekaboni, Danny Eytan, and Anna Goldenberg. Unsupervised Representation Learning for Time Series with Temporal Neighborhood Coding. In *International Conference on Learning Representations*, 2021.

[22] Emadeldeen Eldele, Mohamed Ragab, Zhenghua Chen, Min Wu, Chee Keong Kwoh, Xiaoli Li, and Cuntai Guan. Time-Series Representation Learning via Temporal and Contextual Contrasting. In Zhi-Hua Zhou, editor, *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, pages 2352–2359. International Joint Conferences on Artificial Intelligence Organization, 8 2021. Main Track.

[23] Zhihan Yue, Yujing Wang, Juanyong Duan, Tianmeng Yang, Congrui Huang, Yu Tong, and Bixiong Xu. TS2Vec: Towards Universal Representation of Time Series. In *AAAI Conference on Artificial Intelligence*, 2021.

[24] Tania Pereira, Nate Tran, Kais Gadhoumi, Michele M Pelter, Duc H Do, Randall J Lee, Rene Colorado, Karl Meisel, and Xiao Hu. Photoplethysmography based atrial fibrillation detection: a review. *NPJ Digit. Med.*, 3(1):3, January 2020.

[25] Maximilian Xiling Li, Korbinian Franz Rudolf, Paul Mattes, Nils Blank, and Rudolf Lioutikov. An Overview of Prototype Formulations for Interpretable Deep Learning, 2026.

[26] Chaofan Chen, Oscar Li, Chaofan Tao, Alina Jade Barnett, Jonathan Su, and Cynthia Rudin. *This looks like that: deep learning for interpretable image recognition*. Curran Associates Inc., Red Hook, NY, USA, 2019.

[27] Jonathan Donnelly, Alina Jade Barnett, and Chaofan Chen. Deformable ProtoPNet: An Interpretable Image Classifier Using Deformable Prototypes. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10255–10265, 2021.

[28] Mikołaj Sacha, Dawid Rymarczyk, Łukasz Struski, Jacek Tabor, and Bartosz Zieliński. ProtoSeg: Interpretable Semantic Segmentation with Prototypical Parts, 2025.

[29] Chiyu Ma, Jon Donnelly, Wenjun Liu, Soroush Vosoughi, Cynthia Rudin, and Chaofan Chen. Interpretable image classification with adaptive prototype-based vision transformers. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc.

[30] Yingfan Wang, Haiyang Huang, Cynthia Rudin, and Yaron Shaposhnik. Understanding How Dimension Reduction Tools Work: An Empirical Approach to Deciphering t-SNE, UMAP, TriMap, and PaCMAP for Data Visualization. *Journal of Machine Learning Research*, 22(201):1–73, 2021.